

Congress Trading Signal

La couche données

Extraction, identification, enrichissement et validation honnête des déclarations
boursières du Congrès américain — Chambre et Sénat, 2020–2026

Alice Lemaire · 27 juin 2026

Résumé

Les membres du Congrès américain déclarent leurs transactions boursières sous le *STOCK Act* ; ces déclarations, exploitées par une stratégie de *copy-trading*, exigent au préalable une couche données propre, traçable et honnêtement validée. Ce rapport raconte la construction de cette couche, étape par étape — de la résolution d'identité jusqu'à la table finale enrichie. Le pipeline reconstruit **90 487 transactions** sur deux chambres et sept ans (2020–2026) : **81 646** pour la Chambre (32 676 électronique + 48 970 OCR) et **8 841** pour le Sénat (7 161 électronique + 1 680 papier OCR). Chaque transaction est rattachée à son auteur (**99,99 %** des lignes Chambre, **100 %** au Sénat), enrichie (ticker, secteur **GICS**→**ETF SPDR**, type d'actif, montant *midpoint*, ancienneté), aboutissant à la **table 12/12 champs**. La validation externe contre Quiver — **jamais réinjectée** — donne une concordance transaction-niveau de **85,9 %/an** (Chambre, un plancher) et **98–100 %/an** (Sénat), avec **9** et **0** vrais-absents après explication ; le papier scanné, invisible aux agrégateurs, est une valeur ajoutée propre, validée en interne. La correction est figée par un **golden octet-à-octet** (108 fichiers Chambre, 69 Sénat) et des reproductions fonction-par-fonction. Chaque étape est exposée selon le même fil : le **problème** concret, la **façon dont il est résolu** (avec la fonction réelle), et le **résultat chiffré**.

1 Introduction & contexte

Depuis le **STOCK Act** (2012), les membres du Congrès américain doivent divulguer publiquement leurs transactions sur titres dans un *Periodic Transaction Report* (PTR), en principe sous **45 jours**. L'intuition de recherche — documentée académiquement (Ziobrowski *et al.* 2004/2011, Karadas 2019) — est que ces personnes, par leur appartenance à des commissions clés (Finance, *Armed Services*, Intelligence) ou leur exposition à l'information, produisent par leurs déclarations un **signal potentiellement exploitable**. Une stratégie de *copy-trading* consiste à répliquer ces transactions *après* leur publication (anti-*look-ahead* : on date chaque transaction par sa date de divulgation, jamais par une date postérieure connue).

Avant toute stratégie, il faut une **couche données** irréprochable : extraire fidèlement les déclarations des deux chambres, les rattacher à la bonne personne, les enrichir (ticker, secteur, montant) et les valider honnêtement. C'est l'objet de ce rapport. La stratégie et le backtest associés constituent un travail de recherche séparé, hors du périmètre traité ici.

2 Sources & acquisition

Principe anti-look-ahead. Chaque transaction est datée par sa **date de divulgation** (*disclosure_date*, la date de dépôt du PTR), jamais par une date postérieure connue : une stratégie ne peut copier qu'*après* publication.

Chambre — House Clerk. Un index XML par année (*{an}FD.xml*) liste les dépôts ; on filtre *FilingType=P* (les PTR) et on télécharge le PDF de chaque dépôt. Source publique, sans barrière. Les PDF se répartissent en **lisibles** (couche texte → parsing déterministe) et **scannés** (image → OCR).

Sénat — eFD (*Electronic Financial Disclosure*). Plus fermé : un **garde-fou CSRF**. Il faut

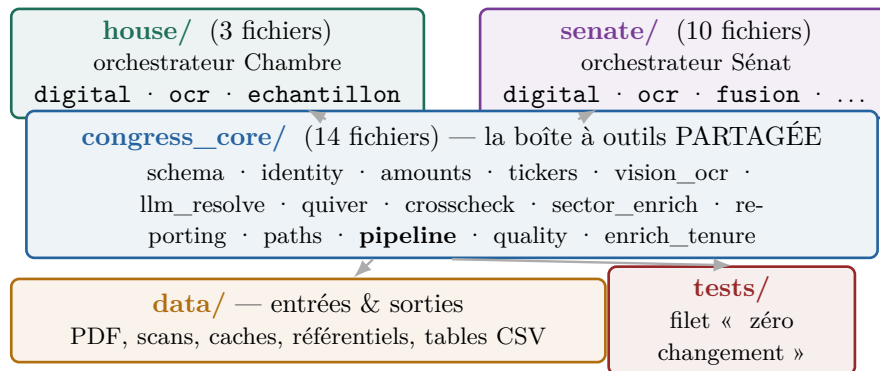
d'abord accepter un formulaire d'accord (`accept_agreement` : GET pour un `csrftoken`, puis POST `prohibition_agreement` avec `csrfmiddlewaretoken` et l'en-tête `X-CSRFToken`). On interroge ensuite l'API de recherche (`report_types=[11]` = PTR). Les déclarations électroniques sont en **HTML** ; les déclarations papier renvoient des **images .gif** servies par `efd-media-public.senate.gov`. On reste « poli » (pause entre requêtes, aucun contournement) et on **cache tout sur disque** — un re-run ne re-télécharge rien.

Quiver = vérification externe *uniquement*. Quiver Quantitative (agrégateur commercial) sert de **vérité-terrain indépendante**. Il n'est *jamais* réinjecté dans nos tables : on mesure si notre extraction concorde, on ne la complète pas. C'est une règle structurante du projet.

3 Architecture & méthodologie

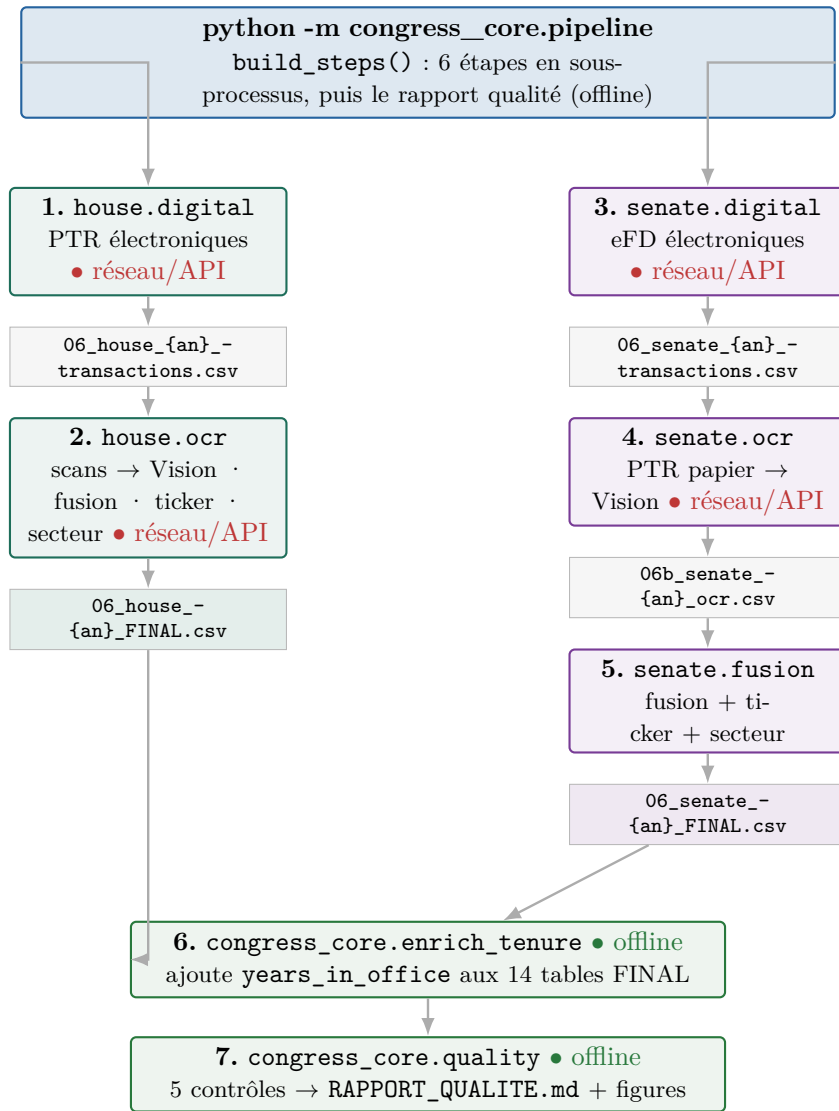
3.1 Trois couches de code + une couche de données

La structure est une **boîte à outils partagée** (`congress_core`, 14 modules) que **deux orchestrateurs** (un par chambre) utilisent, plus la donnée et les tests.



3.2 Le pipeline : un point d'entrée unique

`congress_core/pipeline.py` (fonction `build_steps`) enchaîne les modules en sous-processus. La commande `python -m congress_core.pipeline -years 2020-2026` assemble **6 étapes** d'orchestration ; un **7^e** traitement — le rapport qualité (`congress_core/quality.py`) — se lance séparément, hors-ligne et en lecture seule.



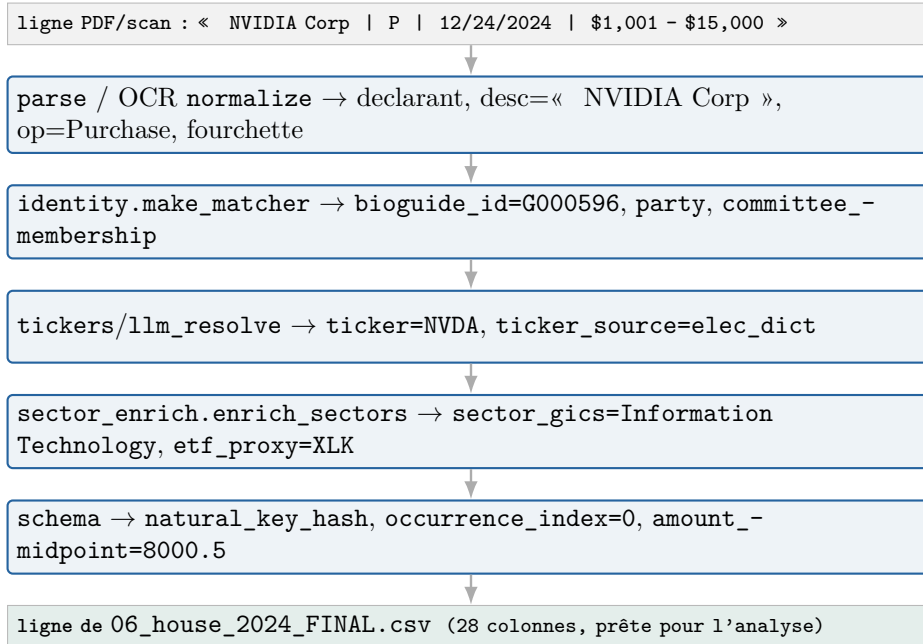
3.3 La convention de numérotation *est* l'enchaînement

Sur le disque, le préfixe d'un fichier = son étape dans le pipeline. C'est ce qui rend la couche données lisible sans documentation externe.

Préfixe	Étape	Écrit par
03_	index des PTR (FilingType=P) dans la fenêtre	house.digital
04_	manifeste : lisible / non_lisible / absent	house.digital
05_	échecs de parsing	*.digital
06_	table digitale (PTR électroniques)	*.digital
06b_	table OCR (scans / papier)	*.ocr
06c_	échecs OCR	*.ocr
06_..._FINAL	digital + OCR fusionnés , 28 colonnes	house.ocr / senate.fusion
07_...07f_	comparaison / réconciliation Quiver (validation)	quiver / quiver_- audit
08_	cross-check « semaine 1 » (Chambre)	house.digital
00_	tableaux de bord (statut, concordance)	*.digital / fusion
_préfixe	caches & brouillons (_quiver_*_cache, _scan_census...)	divers

3.4 Le voyage d'une transaction (vue compacte)

Avant le récit détaillé (§4), voici la trajectoire d'une ligne, de la source à la table finale — en nommant la fonction à chaque étape. C'est elle qui servira de **fil rouge**.



4 Construction de la donnée, étape par étape

Cette section suit une transaction *dans l'ordre où le pipeline la construit* — du nom griffonné sur un formulaire jusqu'à la ligne enrichie de la table finale. Pour chaque étape : le **problème** concret, **comment on le résout** (avec la fonction réelle), et le **résultat** chiffré. Les déclarations arrivent sous **quatre formes** (deux chambres × deux pistes) : **Chambre digitale**, **Chambre OCR**, **Sénat digital**, **Sénat OCR**. Pour chacune, un encart « dans la source » montre la **ligne typique** qu'on y lit, et son **aperçu visuel** (à quoi ressemble la plateforme) figure en annexe 12.1. Ce qui est **commun** aux quatre — l'identité, la clé/dédup, l'enrichissement — n'est expliqué qu'*une* fois.

4.1 D'abord, savoir qui a déclaré : la résolution d'identité

Le problème. Une déclaration ne donne qu'un *nom libre* — « Hon. Earl L. Carter » ici, « Buddy Carter » là, « N. Pelosi » ailleurs. Le même élu apparaît sous plusieurs graphies au fil des années ; sans clé stable, on le compterait deux fois et l'on ne pourrait rattacher ni son parti ni ses commissions.

Comment on le gère. Le module `congress_core/identity.py` charge le référentiel public *congress-legislators* (`load_reference`, en ligne avec repli sur des YAML embarqués), et en tire un index de noms volontairement *tolérant* : pour chaque élu, on indexe non seulement (nom, prénom) mais aussi le **surnom** (table de 31 équivalences : richard→rick, william→bill...), le second prénom, le premier mot du nom officiel et le dernier mot d'un nom composé. Le cœur est la cascade `make_matcher`, qui résout (nom, prénom) dans un ordre de priorité décroissante : (1) un **override manuel** pour les cas irréductibles (un seul est figé : « Nicholas V. Taylor » = Van Taylor) ; (2) la **correspondance exacte** sur l'index ci-dessus (surnoms et seconds prénoms compris) ; (3) à défaut, le **repli par nom de famille unique** (si un seul élu porte ce nom). `enrich_identity` applique le matcher puis ajoute `party`, `committee_membership` et le drapeau de commission clé ; `add_years_in_office` calculera plus tard l'ancienneté. Côté Sénat, `senate/identity.py` ajoute une désambiguïsation par chambre/titulaire et **exclut les délégués** non-votants (sans quoi ils apparaîtraient « chez Quiver mais pas chez nous »).

Le résultat. **99,99 %** des lignes Chambre (81 642 / 81 646) et **100 %** du Sénat reçoivent un `bioguide_id`. Les 4 lignes sans identifiant sont *une* déclaration manuscrite de 2023 (« Ada Norah Henriquez », doc 8219483). On compte **256 bioguides pour 275 noms** (Chambre) : l'écart, ce sont précisément

les variantes orthographiques d'un même élu, ramenées au même identifiant — exactement ce que la résolution corrige.

4.2 Ensuite, trier les formats : électronique ou scanné ?

Le problème. Une déclaration arrive sous deux formes très différentes, qui n'appellent pas le même traitement : un **texte** exploitable directement (on peut le *parser*), ou une **image** qu'il faut « lire » par OCR. Se tromper de piste, c'est soit perdre des données, soit gaspiller des appels Vision sur un document déjà lisible.

Comment on le gère. Côté Chambre, `house/digital.py:build_manifest` ouvre chaque PDF avec `pdfplumber` et applique un test binaire *simple et déterministe* — `bool(text.strip())` : une couche de texte non vide $\Rightarrow$ **lisible** (piste électronique) ; vide $\Rightarrow$ **non_lisible** (piste OCR) ; fichier absent $\Rightarrow$ **absent**. Le verdict est écrit dans `04_download_manifest.csv` et **pilote tout le reste** ; sa simplicité même fait sa fiabilité (pas d'heuristique floue). Côté Sénat, la distinction vient de la liste eFD elle-même (`fetch_ptr_list`) : chaque dépôt porte un `kind` — `ptr` (déclaration HTML électronique) ou `paper` (image .gif scannée).

Le résultat. Ce tri d'apparence anodine est la fourche du pipeline : il explique la composition finale (Chambre 32 676 électronique + 48 970 OCR ; Sénat 7 161 électronique + 1 680 papier) et oriente chaque document vers la bonne des deux sous-sections suivantes.

4.3 La piste digitale de la Chambre (PDF lisibles)

Le problème. Sur un PDF Clerk lisible, les transactions ne forment pas un tableau propre : une *même* transaction peut s'étaler sur plusieurs lignes, le montant déborder sur la ligne suivante, le ticker se cacher entre parenthèses et le libellé de l'actif courir sur des lignes de continuation. Un simple « découpage par ligne » raterait la moitié des transactions.

Comment on le gère. `house/digital.py:run_year` orchestre la séquence : `load_ptr_index` lit l'index XML (`FilingType=P`, fenêtre de divulgation) et produit `03_ptr_index.csv` ; `parse_ptr` fait le gros du travail — il **reconstitue** d'abord les lignes coupées (`_join_split_lines`, qui recolle un montant scindé en deux), détecte chaque transaction par l'expression régulière `TXN_RE` (un code P|S|E, deux dates, une fourchette \$), puis remonte de quelques lignes pour rattacher la description d'actif et le ticker (entre parenthèses ou préfixe d'échange). `join_identity` rattache ensuite le bioguide et les métadonnées, et `finalize` calcule la clé naturelle, l'`occurrence_index` et déduplique. La sortie est `06_house_{an}_transactions.csv` ; les PDF lisibles ne donnant *aucune* ligne sont tracés dans `05_parse_failures.csv` — un échec est toujours visible, jamais silencieux.

Le résultat. **32 676** transactions électroniques, avec un taux de parsing très élevé (99–100 %).

Dans la source. Un PTR électronique de la Chambre se lit comme un tableau propre (cf. annexe 12.1) :
« Amazon.com, Inc. — Common Stock (**AMZN**) [ST] · Purchase · 03/16/2018 · \$1 001–\$15 000 ». Le ticker est entre parenthèses, le type d'actif entre crochets — d'où le parsing déterministe.

4.4 La piste digitale du Sénat (HTML eFD)

Le problème. Le Sénat n'expose pas de PDF : il faut d'abord franchir un **garde-fou CSRF** (un jeton qu'on n'obtient qu'en acceptant un formulaire d'accord), puis parser des **tableaux HTML** dont l'intitulé et l'ordre des colonnes *varient* d'une déclaration à l'autre.

Comment on le gère. `senate/digital.py:run_year` commence par `accept_agreement` (GET du `csrftoken`, POST `prohibition_agreement` avec l'en-tête `X-CSRFToken`), puis `fetch_ptr_list` interroge l'API de recherche (`report_types=[11]`) et `fetch_report` récupère chaque déclaration (avec cache disque). `parse_electronic` lit alors les tableaux via `pd.read_html` — qui renvoie une liste de *DataFrames* bruts — et c'est `_find_col` qui **retrouve les bonnes colonnes par appariement flou** (chercher une colonne dont le nom contient « ticker », ou « asset » et « name »), seule façon robuste face à des en-têtes irréguliers. L'opération est normalisée (`norm_op`) et le montant ramené à son milieu (`amount_midpoint`) ; l'identité, le ticker et la déduplication sont délégués à `senate/identity.py:enrich`. Sortie : `06_senate_{an}_transactions.csv`.

Le résultat. 7 161 transactions électroniques pour le Sénat.

Dans la source. Une déclaration eFD du Sénat est une page web (cf. annexe 12.1) : « **YUM** · Yum! Brands (Yum, Louisville KY) · Sale (Full) · \$1001–\$15 000 · Joint », avec le ticker cliquable et la date de transaction — d'où la lecture par `pd.read_html`.

4.5 La piste scannée de la Chambre (OCR Claude Vision)

Le problème. Reste l'autre moitié : des **images**. À quoi ressemblent-elles ? Le census des 547 PDF scannés en distingue trois familles : **A** — **tapé droit** (74 docs), **B** — **tapé mais l'image est couchée** (322 docs, le gros du volume), **C** — **manuscrit** (151 docs). Or un modèle de vision lit mal une page tournée, et plus mal encore une date manuscrite.

Comment on le gère. Le moteur `congress_core/vision_ocr.py` **redresse d'abord**. L'idée naïve — demander au modèle « de combien faut-il tourner ? » — échoue (il répond presque toujours « 90 ») ; à la place, `detect_rotation` lui montre la page aux **4 orientations** (`ROT_CANDIDATES=[0,90,180,270]`) et lui fait *reconnaître* celle qui est droite — une tâche de reconnaissance, bien plus fiable qu'une estimation d'angle — puis `rotate_b64_png` la remet droite. `VisionExtractor._call` extrait ensuite en JSON structuré (`tool_choice` forcé sur `record_transactions`, modèle `claude-sonnet-4-6`, 7 tentatives), avec un **cache versionné** par `prompt_sha` (`extract_cached`) qui ne re-paie que les lots en erreur. Le ticker, absent des formulaires papier, est **ré-associé** en propageant les symboles vus en électronique, complété par une passe LLM (`house/ocr.py:llm_resolve_tickers`). **Point d'architecture** : côté Chambre, la **fusion digital+OCR, le ticker et le secteur se font ANNÉE PAR ANNÉE**, pendant le run (`house/ocr.py:run_ocr_year` écrit directement `06_house_{an}_FINAL.csv`), parce que le volume y est énorme. Le cluster C manuscrit est **exclu par défaut** (dates trop incertaines pour une stratégie datée), avec récupération ciblée des gros déposants.

Le résultat. 48 970 lignes OCR. Le volume est très concentré : Rohit Khanna en représente **62,9 %** à lui seul, le top 3 (Khanna, McCaul, Harshbarger) **92,4 %** — ce sont de gros déposants qui déclarent exclusivement sur papier.

Dans la source. Un scan de la Chambre est un **formulaire à cases** (cf. annexe 12.1), souvent *couché* : une transaction = une ligne où l'on coche le type (Purchase/Sale), la date, et la **fourchette de montant** (une colonne à cocher parmi plusieurs) — d'où le besoin de deskew puis de Vision pour la lire.

4.6 La piste papier du Sénat — et pourquoi la fusion y est séparée

Le problème. Côté Sénat, le papier est marginal (5 sénateurs) mais bien réel : des images `.gif`. Surtout, contrairement à la Chambre, les données sont *peu nombreuses* — et un dictionnaire de tickers construit année par année y serait trop pauvre.

Comment on le gère. `senate/ocr.py` lit les `.gif` par Vision et écrit `06b`. Puis — et c'est là l'**unique vraie asymétrie d'architecture** entre les deux chambres — `senate/fusion.py:main` procède en deux temps : **Étape 1**, fusion non destructrice *par année* (digital + OCR, dédup sur (`natural_key_hash`, `occurrence_index`)) ; **Étape 2**, enrichissement sur le **corpus complet en une seule passe** — `ticker_resolve.resolve_tickers` puis `enrich_sectors` sur *toutes* les années concaténées, avec « dict + LLM partagés, cache unique ». **Pourquoi cette différence avec la Chambre ?** La Chambre a assez de volume pour traiter chaque année isolément ; le Sénat, lui, est petit et chargé en papier — **mutualiser le dictionnaire de tickers sur tout le corpus** (un symbole vu une année sert aux autres) relève nettement la couverture. D'où un module `fusion.py` dédié, exécuté *après* l'assemblage de toutes les années, là où la Chambre fonde et enrichit *pendant* chaque année.

Le résultat. 1 680 lignes papier, très concentrées sur Blumenthal (~73 %). C'est ce papier (obligations municipales, *munis*) qui explique la baisse de couverture ticker/secteur du Sénat en 2025–26 (ğ5).

Dans la source. Le papier du Sénat est un formulaire scanné (cf. annexe 12.1) : ex. « SFA Partners LLC · Purchase · 11/4/05 » ou « Microsoft, NASDAQ/OTC · 2/17/10 », avec la fourchette de montant en colonnes cochées — comme la Chambre OCR, mais sans symbole boursier imprimé (à ré-associer).

4.7 Donner un secteur à chaque ticker (GICS → ETF SPDR)

Le problème. Une stratégie en ETF a besoin du **secteur**, pas seulement du ticker ; or certains tickers sont délistés, fusionnés ou introuvables dans les bases standard.

Comment on le gère. `congress_core/sector_enrich.py:enrich_sectors` résout en cascade hybride : `resolve_yfinance` (secteur factuel via `yfinance`) → `repli resolve_llm` (énumération GICS stricte, pour les tickers que `yfinance` ignore) → overrides manuels, en **traçant qui a tranché** (`sector_source`). Le secteur GICS est mappé 1 : 1 vers un **ETF Select Sector SPDR** (Energy XLE, Materials XLB, Industrials XLI, Cons. Disc. XLY, Cons. Staples XLP, Health Care XLV, Financials XLF, IT XLK, Comm. Services XLC, Utilities XLU, Real Estate XLRE). Au passage, les montants déclarés en **fourchettes** sont ramenés à leur **milieu** (`amounts.py:amount_midpoint`) ; et l'ancienneté `years_in_office` est calculée hors-ligne puis *appendue* en dernière étape (`enrich_tenure.py`, octet-préservant et idempotent).

Le résultat. Secteur renseigné à **83,2 %** (Chambre) et **62,1 %** (Sénat) ; ancienneté à **100 %** sur les deux chambres.

4.8 La table finale : le contrat « 12/12 »

Le liant — le cœur partagé. Ce qui assemble tout sans rien perdre, c'est `schema.py` : une **clé naturelle** de 7 champs (`chamber`, `declarant_name`, `transaction_date`, `asset_description`, `operation_type`, `amount_range`, `owner`). Elle exclut *volontairement* le ticker — qui est ajouté *après*, à l'enrichissement ; l'inclure rendrait la clé instable. Cette clé est numérotée par `occurrence_index` (le rang d'un lot répété *dans un même dépôt*), si bien que la déduplication (`dedup_canonical`) retire les vrais doublons cross-dépôt mais **préserve les lots multi-comptes réels** — par exemple les déclarations répétées de Khanna via plusieurs comptes, qui sauteraient avec une dédup naïve.

Le contrat. La table FINAL compte **28 colonnes** (27 d'assemblage + `years_in_office`). Les **12 champs « métier »** exigés sont *exactement* la constante `schema.CORE_FIELDS` :

```
declarant_name · chamber · party · committee_membership · transaction_date · disclosure_date  
· ticker · sector_gics · etf_proxy · operation_type · amount_midpoint · asset_type
```

Nuance d'honnêteté — « présents » n'est pas « sans valeurs manquantes ». Les 12 champs sont **présents** dans le schéma des deux chambres. Mais trois ont des **trous légitimes**, par nature et non par défaut d'extraction : `ticker/sector_gics/etf_proxy` sont vides pour les actifs non cotés (obligations, *munis*, fonds privés — il n'existe pas de symbole à leur attribuer) ; et `committee_membership` est un **snapshot courant**, pas l'historique daté. Les champs structurellement toujours renseignables (nom, chambre, parti, dates, opération, montant) sont, eux, quasi-complets. Nous assumons cette distinction plutôt que de la masquer. Les taux de remplissage exacts (par champ et par an) sont en annexe 12.4.

5 Le rapport de qualité

Le problème. Une table « pleine » n'est pas une table *saine* : les dates peuvent être incohérentes, les délais légaux non respectés, la couverture concentrée sur quelques personnes.

Comment on l'obtient. `congress_core/quality.py (build_report → _figures)` calcule **5 contrôles** et génère 4 figures — **en lecture seule des FINAL, sans aucun appel API** (matplotlib Agg). (a) **Cohérence des dates** (`disclosure ≥ transaction`) : 99,8 % Chambre, 100,0 % Sénat. (b) **Délai légal** : 86,9 % (Chambre) et 84,9 % (Sénat) des dépôts dans les 45 jours (médiane **28 j**). (c) **Montants** : médiane **8 000 \$**, quartile haut **32 500 \$**. (d) **Couverture par membre** : 320 déposants, dont **206** avec ≥ 10 transactions (éligibles) et **150** actifs sur ≥ 3 années. (e) **Achats sans sortie déclarée** : 22,5 % (Chambre) / 39,6 % (Sénat). Le détail par an est en annexe 12.4.

La couverture par an — et la baisse Sénat 2025–26. Le global masque une dynamique : au Sénat, la couverture ticker chute à **45,4 %** (2025) et **43,7 %** (2026), contre $\sim 80\%$ les années antérieures ; le secteur suit. **Pourquoi ?** La part de papier OCR et surtout d'**obligations municipales** (non cotées,

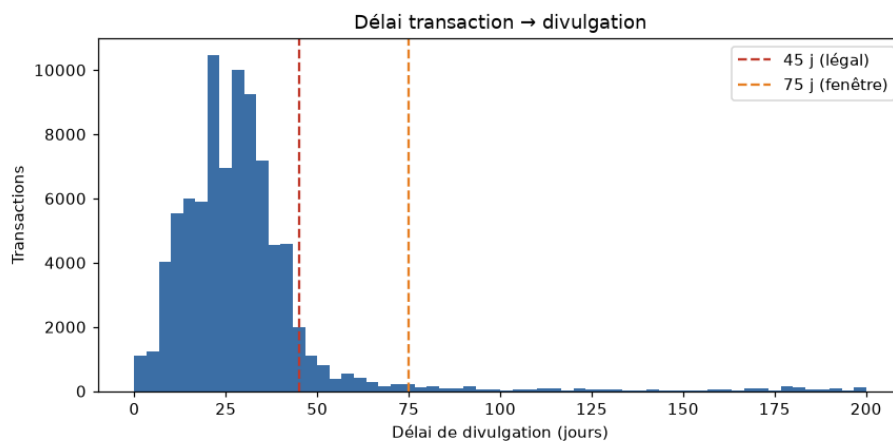


FIGURE 1 – Délai disclosure – transaction (0–200 j), lignes 45 j (légal) / 75 j (fenêtre). Générée par `congress_core/quality.py` (`_figures`, source `delay_buckets`) — lecture seule, aucun appel API.

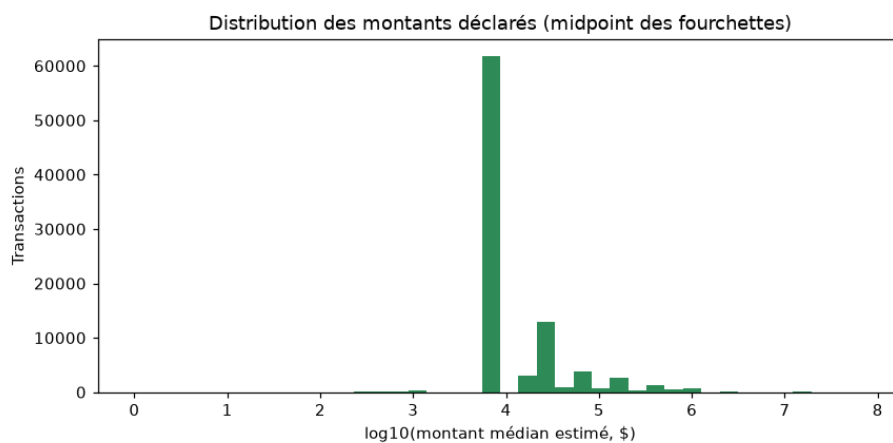


FIGURE 2 – Distribution de `amount_midpoint` en échelle `log10`. Générée par `quality.py` (`_figures`, source `amount_distribution`).

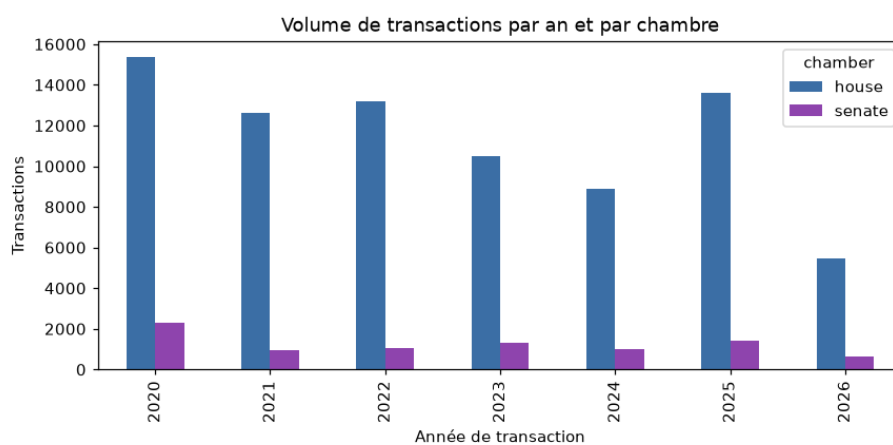


FIGURE 3 – Transactions par **année de transaction** et par chambre. Générée par `quality.py` (`_figures`).

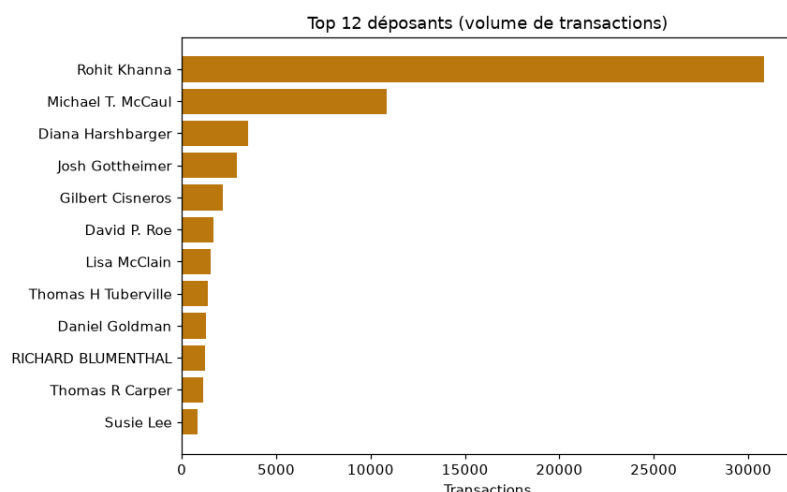


FIGURE 4 – Top 12 déposants par **nombre de transactions**. Générée par `quality.py` (`_figures`, source `coverage_per_member`) — la domination des déposants-papier (Khanna, McCaul) y est visible.

donc `ticker=None` *légitimement*, ex. Blumenthal) y augmente fortement. Ce n'est pas une dégradation d'extraction mais un changement de composition des actifs ; le global (71,4 % / 62,1 %) est tiré vers le haut par les années antérieures. La fiabilité des dates OCR est suivie par `date_confidence` (fenêtre 75 j, colonne *OCR seulement*) : 95,3 % plausibles côté Chambre — le creux 2021 (79,6 %) est entièrement Diana Harshbarger (manuscrit dense), anomalie localisée, non systémique.

6 La validation externe (Quiver)

Le problème. Comment savoir si l'extraction est juste, *sans* se contenter de se croire ?

Comment on le gère. On compare le FINAL à Quiver sur **deux axes**, sans jamais le réinjecter : (1) **comptes par déposant** (per-lot) ; (2) **recouvrement transaction-niveau** via la clé bioguide | ticker | date | type (la date appariée est le *Traded* de Quiver). La **couverture** = transactions Quiver retrouvées / total Quiver.

Le résultat — **Chambre** (clé « brute », sans tolérance de date, `quiver.validate_quiver_house`) : couverture globale **85,9 %** (43 136 / 50 224), par année de 80,7 % à 91,3 %. C'est un **plancher** : la clé sur date brute sous-estime sur les dates OCR bruitées. Les **249 « manquants bruts »** se réduisent à **9 vrais-absents** après explication des artefacts (tolérance ± 1 j, amendements, options/obligations) — **aucun bug d'extraction**. **Sénat** (clé normalisée, `quiver_audit.reconcile`) : couverture **98–100 %/an**, accord de sens $\sim 95,7$ %, montant $\sim 93,2$ %, **554 amendements** dédoublés, **0 vrai raté** après audit adversarial. Le détail par an est en annexe 12.4.

Le constat « OCR = source unique ». Quiver — comme Kadoa et *House Stock Watcher* — **ne voit aucun scan** : 0 ligne pour les gros déposants-papier (Khanna, Harshbarger, Blumenthal). Le terme `only_ours` de la Chambre ($\sim 16\,000$ transactions) est donc surtout le **papier que les agrégateurs ne voient pas** — pas une erreur, mais notre **valeur ajoutée**, validée en interne (cohérence, stabilité run-à-run, `date_confidence`). `crosscheck.py` matérialise ce statut par déposant (`quiver_validable / ocr_unique / digital` ; détail en annexe 12.4).

7 Assurance qualité & reproductibilité

Le problème. Le pipeline n'est **pas rejouable hors-ligne** (le scraping exige le réseau, l'OCR l'API) ; rejouer une année digitale produirait une table *vide* (les PDF lisibles ont été parsés puis écartés).

Comment on le gère. On prouve la correction par **reproduction depuis les colonnes figées**, preuve plus forte car déterministe : (i) un **golden octet-à-octet** — empreintes SHA-256 de toutes les

sorties, **108** fichiers Chambre et **69** Sénat (`check_golden.py` / `senate_check_golden.py` doivent afficher « ZÉRO ÉCART »); (ii) des **reproductions fonction-par-fonction** (`test_senate_repro` reconstruit `natural_key_hash` 8841/8841, l'identité 67/67, le ticker 65/65; côté Chambre `test_schema/test_amounts_tickers/` `test_identity/test_tenure`); (iii) `test_vision_sha/test_incremental` (le `prompt_sha` est stable, un 2^e run OCR fait 0 appel); (iv) `audit_metrics.py`, qui recompute toutes les métriques et *assure* les invariants — la source de vérité chiffrée de ce rapport.

8 Hypothèses explorées puis révisées

La valeur « recherche » tient autant aux choix *révisés* qu'aux résultats. Pour chacun : **hypothèse de départ** → **problème constaté** → **décision finale**.

Échantillon d'abord. Avant le run complet des 547 PDF, `house/echantillon.py` a OCR-isé un **échantillon stratifié** des clusters A/B/C pour *mesurer* la qualité et durcir le prompt — plutôt que de découvrir les modes d'échec en production.

Deskew OCR. *Départ* : demander au modèle « de combien faut-il tourner ? ». *Problème* : tâche spatiale peu fiable — il répond presque toujours « 90 ». *Décision* : lui montrer la page aux **4 rotations** et lui faire *reconnaître* la droite. La concordance OCR passe de **59,7 %** à **≈69 %**.

Déduplication. *Départ* : dédupliquer sur la seule clé naturelle. *Problème* : cela écrase des **lots multi-comptes réels** (Khanna déclarant plusieurs fois le même titre). *Décision* : `occurrence_index` → **dédup non destructrice** sur le couple.

Cluster C manuscrit. Tenté, puis **exclu** par défaut (dates trop incertaines pour une stratégie datée). Une sonde **Opus vs Sonnet** sur C a confirmé qu'Opus **ne corrige pas le goulot** (les dates), pour un coût $\approx \times 3$ — on garde Sonnet, C reste exclu.

Fenêtre de plausibilité. Le STOCK Act vise ≈ 45 j ; on tolère **75 j** (`date_confidence`) pour absorber le bruit OCR — colonne calculée *sur l'OCR seulement*.

Ticker en trois voies. Ajoutées par échecs successifs : explicite, dictionnaire électronique, puis LLM (filtre `is_equity`, pour ne *jamaïs* tickeriser une obligation/muni).

Quiver, deux régimes de clé. Chambre = clé « brute » (couverture = plancher); Sénat = clé « normalisée » (*reconcile*) avec dédup des amendements. On *n'unifie pas* les clés : chaque chambre garde sa sémantique exacte.

Per-année (Chambre) vs corpus (Sénat). L'enrichissement se fait *pendant* chaque année côté Chambre (volume), mais *sur tout le corpus en une passe* côté Sénat (pour mutualiser le dictionnaire de tickers) — cf. §4.

9 Limites (honnêteté)

- **Cluster C manuscrit exclu par défaut** (dates trop incertaines) — choix assumé, pas une lacune cachée.
- **Validation papier externe impossible** : aucun agrégateur ne voit les scans → pas de taux d'exactitude *externe* sur le papier (seulement des contrôles internes).
- **Appariement Quiver Chambre sans tolérance de date** : la couverture **85,9 %** est un **plancher**, pas une borne haute.
- `ticker/sector_gics/etf_proxy` **structurellement absents** pour munis et obligations (actifs non cotés) — par nature, pas par défaut.
- **Commissions non « point-in-time »** : `committee_membership` est un *snapshot* courant, pas l'historique daté.
- `date_confidence` = **heuristique binaire** (fenêtre 75 j), **colonne OCR seulement** ; ce n'est pas une vérité-terrain.

10 Périmètre & suite

La **couche données** décrite ici est livrée : 90 487 transactions extraites, identifiées, enrichies (table 12/12), validées et reproductibles. La **stratégie et le backtest** associés (copy-trading actions puis ETF sectoriels) constituent un travail de recherche séparé, hors-périmètre de ce rapport.

11 Conclusion

Sur deux chambres et sept ans, le pipeline reconstruit **90 487** transactions à partir des sources officielles, en traitant honnêtement les deux pistes (électronique déterministe *et* scannée par OCR Vision). Chaque ligne est identifiée (99,99 / 100 %), enrichie (table 12/12 sur les deux chambres) et validée contre Quiver sans jamais le réinjecter (85,9 % Chambre, 98–100 % Sénat ; 9 et 0 vrais-absents). Le papier scanné, invisible aux agrégateurs, est une valeur ajoutée propre, validée en interne ; la correction est figée (golden octet-à-octet) et reproductible. Surtout, le rapport assume ses nuances — « présents » n’est pas « sans manquants », la couverture Sénat 2025–26 baisse pour des raisons de composition d’actifs, et la couverture Quiver Chambre est un plancher. C’est une couche données sur laquelle une stratégie peut être bâtie *en connaissant ses limites*.

12 Annexes

12.1 Annexe A — aperçu des sources

À quoi ressemble la matière première des **quatre pistes** (deux chambres × digital/OCR). Captures des plateformes publiques (*House Clerk*, Sénat *eFD*) et rendu d'un scan du dépôt.

PERIODIC TRANSACTION REPORT
Clerk of the House of Representatives • Legislative Resource Center • 8th Cannon Building • Washington, DC 20515

K INFORMATION
1 Hon. Mark Allen
4 Member
6 District: MD04

SACTIONS

Owner Asset	Transaction Date Type	Notification Amount Date	Cap. Gains > \$200?
Amazon.com, Inc. - Common Stock (AMZN) [ST]	S (partial)	03/16/2025 03/16/2025	\$1,001 - \$15,000
Private Source: New Securities On: Putman Investments Description: The full transaction included the following sales: T - 37,428 shares sold @ \$27.645/share BKC/B - 3 shares sold @ \$493.415/share SPY - 8,318 shares sold @ \$676.014/share DIA - 8,420 shares sold @ \$470.056/share SPYD - 37,070 shares sold @ \$47.014/share AMZN - 23 shares sold @ \$189.401/share AAPL - 20,252 shares sold @ \$323.432/share QQQ - 4,415 shares sold @ \$601.684/share PYPL - 9,001 shares sold @ \$47.735/share			
Apple Inc. - Common Stock (AAPL) [ST]	S (partial)	03/16/2025 03/16/2025	\$1,001 - \$15,000
Private Source: New Securities On: Putman Investments Description: The full transaction included the following sales: T - 37,428 shares sold @ \$27.645/share BKC/B - 3 shares sold @ \$493.415/share SPY - 8,318 shares sold @ \$676.014/share DIA - 8,420 shares sold @ \$470.056/share SPYD - 37,070 shares sold @ \$47.014/share AMZN - 23 shares sold @ \$189.401/share AAPL - 20,252 shares sold @ \$323.432/share QQQ - 4,415 shares sold @ \$601.684/share PYPL - 9,001 shares sold @ \$47.735/share			
AT&T Inc. (T) [ST]	S (partial)	03/16/2025 03/16/2025	\$1,001 - \$15,000
Private Source: New Securities On: Putman Investments Description: The full transaction included the following sales: T - 37,428 shares sold @ \$27.645/share BKC/B - 3 shares sold @ \$493.415/share SPY - 8,318 shares sold @ \$676.014/share DIA - 8,420 shares sold @ \$470.056/share SPYD - 37,070 shares sold @ \$47.014/share AMZN - 23 shares sold @ \$189.401/share AAPL - 20,252 shares sold @ \$323.432/share QQQ - 4,415 shares sold @ \$601.684/share PYPL - 9,001 shares sold @ \$47.735/share			
Berkshire Hathaway Inc. New Common Stock (BRK.B) [ST]	S (partial)	03/16/2025 03/16/2025	\$1,001 - \$15,000
Private Source: New			

HAND DELIVERED
03/16/2025 03/16/2025
03/16/2025 03/16/2025

PERIODIC DISCLOSURE OF FINANCIAL TRANSACTIONS

Reporting Individual's Name: Richard Blumenthal
Announcement: Senate Office / Agency or Other Employer: United States Senate

Amount of Transaction (\$)

Transaction Type (S)	Transaction Date (MM/DD/YYYY)	Amount of Transaction (\$)
S	03/16/2025	\$1,001 - \$15,000

(a) **Chambre — digitale.** PTR électronique (*House Clerk*) : tableau texte, ticker entre parenthèses (AMZN), type d'actif entre crochets.

(b) **Chambre — OCR.** PDF scanné, ici *couché* : formulaire à cases, redressé par deskew avant lecture Vision.

UNITED STATES SENATE FINANCIAL DISCLOSURES
Return to the search tab to select another report.

Periodic Transaction Report for 03/10/2025
The Honorable James Banks (Banks, James E.)
Filed 03/10/2025 at 12:54 PM

The following statements were checked before filing:
I certify that the statements I have made on this form are true, complete and correct to the best of my knowledge and belief.
I understand that reports cannot be edited once filed. To make corrections, I will submit an electronic amendment to this report.

Transactions (1 transaction total) 0 Self 1 Joint 0 Spouse 0 Dependent Child

#	Transaction Date	Owner	Ticker	Asset Name	Asset Type	Type	Amount	Comment
1	03/10/2025	Joint	YUMC	Yum Brands Company: Yum (Louisville, KY) Description: Food	Other	Sale (Full)	\$1,001 - \$15,000	--

PERIODIC DISCLOSURE OF FINANCIAL TRANSACTIONS
Secretary of the Senate
Office of Public Records
West Building, Suite 502
Washington, DC 20510

Reporting Individual's Name: Richard Blumenthal
Announcement: Senate Office / Agency or Other Employer: United States Senate

Amount of Transaction (\$)

Transaction Type (S)	Transaction Date (MM/DD/YYYY)	Amount of Transaction (\$)
S	03/10/2025	\$1,001 - \$15,000

(c) **Sénat — digital.** Page web eFD : ticker cliquable (YUM), type d'opération et montant — lue par pd.read_html.

(d) **Sénat — OCR.** Formulaire papier scanné (eFD *media*) : lignes à cocher, fourchettes de montant en colonnes.

12.2 Annexe B — modules & fonctions clés

Pour chaque module : son rôle, et ses **fonctions centrales** (les helpers privés mineurs sont omis). Le pipeline complet vit dans 27 modules de code (3 couches) ; l'orchestration se lit dans `pipeline.py:build_steps`, puis `house/digital.py:run_year`, `house/ocr.py:run_ocr_year`, `senate/digital.py:run_year` et `senate/fusion.py:main`.

Module	Rôle	Fonctions clés
congress_core/	boîte à outils partagée	

Module	Rôle	Fonctions clés
schema.py	clé naturelle + dédup	natural_key_hash (SHA-256 de 7 champs) · add_occurrence_index · dedup_canonical (non destructrice)
identity.py	nom → bioguide + méta	load_reference · make_matcher (override→exact+urnoms→nom unique) · enrich_identity · add_years_in_office
amounts.py	montant & opération	amount_midpoint (milieu de fourchette) · operation_type_from_code (P/S/E)
tickers.py	ticker + type d'actif	explicit_ticker · recover_ticker · infer_asset_type (par chambre)
vision_ocr.py	OCR Vision + cache	detect_rotation (deskew 4 rotations) · pdf_to_b64_images · VisionExtractor.prompt_sha · .extract_cached (reprise au batch)
llm_resolve.py	cache LLM nom→ticker	resolve_names_to_tickers · VersionedLLMCache · map_batch (tool_use + backoff)
quiver.py	validation externe	fetch_quiver · validate_quiver_house (clé brute) · reconcile (clé normalisée)
crosscheck.py	statut par déposant	per_filer_status · add_external_counts (Kadoda/HSW) · summary
sector_enrich.py	secteur GICS → ETF	resolve_yfinance · resolve_llm · enrich_sectors (+ sector_source)
reporting.py	sorties & QA	qa_flags · upsert_status · write_excel
paths.py	chemins & secrets	DataRoot (data/{chambre}) · get_secret / load_env
pipeline.py	orchestrateur	build_steps (séquence 6 étapes) · main (sous-processus, -dry-run)
quality.py	rapport qualité	load_final · date_coherence · delay_buckets · amount_distribution · coverage_per_member · unmatched_purchase_rate · build_report
enrich_tenure.py	ancienneté	enrich_file (append years_in_office octet-préservant) · main
house/ — orchestrateur Chambre		
digital.py	PTR électroniques	load_ptr_index (XML, FilingType=P) · build_manifest (lisible/non) · parse_ptr (TXN_RE) · join_identity · finalize · validate_quiver · run_year
ocr.py	scans → FINAL (par an)	extract_from_pdf (Vision) · llm_resolve_tickers · normalize (+ date_confidence) · run_ocr_year (fusion+ticker+secteur)
echantillon.py	pilote OCR	run_echantillon (stratifié) · compare_quiver
senate/ — orchestrateur Sénat		
digital.py	eFD électronique	accept_agreement (CSRF) · fetch_ptr_list (report_types=[11]) · parse_electronic (pd.read_html) · run_year
ocr.py	papier .gif → 06b	discover_index · ocr_one (un .gif → Vision) · build_table
ocr_engine.py	moteur OCR figé	extract_report · normalize · _fix_year
fusion.py	fusion + enrich corpus	main (Étape 1 fusion/an, Étape 2 enrichissement corpus) · date_confidence
identity.py	référentiel Sénat	load_reference · make_matcher (désambiguïsation chambre) · enrich (bioguide+ticker+dédup+schéma)
ticker_resolve.py	ticker 3 voies	resolve_tickers (explicit→dict→LLM) · llm_resolve_tickers
sector_enrich.py	secteur Sénat	resolve_sectors · enrich_sectors
quiver_audit.py	réconciliation	reconcile (07c-07f)
revalidate_quiver.py	re-validation offline	main
census_probe.py	Phase 0 (census)	parse_electronic · main (census + sondage parser)

Les 3 `__init__.py` (exports de paquet) et `tests/regression/` (golden + reproductions) complètent les 44 fichiers `.py` du dépôt.

12.3 Annexe C — le schéma FINAL (28 colonnes)

#	Colonne	Sens	Remplie par
1	<code>bioguide_id</code>	identifiant officiel du membre	<code>identity</code>
2	<code>declarant_name</code>	nom tel que déposé	<code>parse / OCR</code>
3	<code>chamber</code>	<code>house / senate</code>	<code>enrich_identity</code>
4	<code>party</code>	parti	<code>Reference.party</code>
5	<code>state_district</code>	état + district	<code>03_ptr_index</code>
6	<code>committee_membership</code>	commissions (liste)	<code>Reference.committees</code>
7	<code>committees_key_flag</code>	commission « clé » ?	<code>is_key_committee</code>
8	<code>transaction_date</code>	date de la transaction	<code>parse / OCR</code>
9	<code>disclosure_date</code>	date de divulgation (dépôt)	<code>03_ptr_index</code>
10	<code>ticker</code>	symbole boursier	<code>tickers / llm_resolve</code>
11	<code>asset_description</code>	libellé de l'actif	<code>parse / OCR</code>
12	<code>asset_type</code>	type d'actif	<code>infer_asset_type</code>
13	<code>sector_gics</code>	secteur GICS	<code>sector_enrich</code>
14	<code>etf_proxy</code>	ETF SPDR du secteur	<code>GICS_TO ETF</code>
15	<code>operation_type</code>	Purchase / Sale / ...	<code>operation_type_from_code</code>
16	<code>amount_range</code>	fourchette déclarée	<code>parse / OCR</code>
17	<code>amount_midpoint</code>	milieu de la fourchette	<code>amount_midpoint</code>
18	<code>amount_split_flag</code>	lot multi-actifs ?	<code>finalize</code>
19	<code>owner</code>	propriétaire (self/spouse...)	<code>normalize</code>
20	<code>doc_id</code>	identifiant du PTR	<code>03_ptr_index</code>
21	<code>source_url</code>	URL de la source	<code>03_ptr_index</code>
22	<code>natural_key_hash</code>	clé de dédup (7 champs)	<code>schema</code>
23	<code>occurrence_index</code>	rang du lot intra-dépôt	<code>add_occurrence_index</code>
24	<code>provenance</code>	<code>*-electronic / *-ocr</code>	<code>pipeline</code>
25	<code>date_confidence</code>	plausible / implausible (75 j)	<code>date_confidence</code>
26	<code>ticker_source</code>	explicit / elec_dict / llm / none	<code>tickers / llm</code>
27	<code>sector_source</code>	yfinance / llm / manual	<code>sector_enrich</code>
28	<code>years_in_office</code>	ancienneté à la date du trade	<code>enrich_tenure</code>

L'ordre exact diffère légèrement entre *Chambre* et *Sénat* (*date_confidence* plus tôt au Sénat); le contenu garanti — les 12 champs « métier » — est identique.

12.4 Annexe D — métriques vérifiées (recalculées depuis les FINAL)

Lue après le rapport, cette annexe dit si l'extraction a bien marché. Tous les chiffres sont recomputés depuis les tables FINAL par `tests/regression/audit_metrics.py` et `congress-core/quality.py`.

C.1 — Volumes par an.

An	Chambre				Sénat			
	dig.	OCR	FINAL	dépos.	dig.	OCR	FINAL	sén.
2020	6 886	8 791	15 677	109	1 706	255	1 961	33
2021	5 457	6 816	12 273	120	1 098	281	1 379	35
2022	3 601	10 900	14 501	110	919	182	1 101	27
2023	4 161	6 329	10 490	100	1 062	97	1 159	30
2024	2 694	6 277	8 971	97	946	84	1 030	24
2025	7 577	6 067	13 644	107	943	478	1 421	31
2026	2 300	3 790	6 090	84	487	303	790	22
Tot.	32 676	48 970	81 646	256*	7 161	1 680	8 841	64*

* bioguides *uniques* sur 7 ans (la somme annuelle compte un élu chaque année).

C.2 — Couverture des champs enrichis, par an (% renseigné).

An	Chambre				Sénat			
	ticker	sect.	a.type	comm.	ticker	sect.	a.type	comm.
2020	86,5	83,4	91,0	64,9	88,1	80,5	96,3	34,5
2021	87,3	85,8	90,1	78,7	80,9	70,1	94,2	54,3
2022	87,1	85,2	91,0	89,2	79,7	65,3	96,1	78,3
2023	78,9	76,6	88,6	96,8	77,7	69,5	96,3	71,4
2024	81,3	78,3	87,4	93,6	68,0	59,2	100	77,8
2025	87,8	86,3	94,8	97,6	45,4	37,2	100	84,9
2026	85,4	84,4	94,9	99,7	43,7	35,7	100	86,2
Glob.	85,3	83,2	91,1	86,6	71,4	62,1	97,3	65,6

C.3 — Les 5 contrôles qualité (global).

Contrôle	Chambre	Sénat
(a) dates cohérentes ($\text{disclosure} \geq \text{transaction}$)	99,8 %	100,0 %
(b) dépôts ≤ 45 j (délai médian 28 j)	86,9 %	84,9 %
(c) montants — médiane / quartile haut	8 000 \$ / 32 500 \$	
(d) déposants (≥ 10 trades / ≥ 3 ans)	320 (206 / 150)	
(e) achats sans sortie déclarée	22,5 %	39,6 %

C.4 — Concordance Quiver Chambre, par an (clé brute ; couverture = plancher).

An	couv. %	Quiver retrouvées	only_quiver	vrais-absents
2020	85,0	7 106	1 253	16
2021	88,6	5 221	672	14
2022	80,7	6 061	1 452	29
2023	81,0	6 128	1 442	65
2024	85,8	4 670	772	40
2025	91,3	9 835	938	58
2026	88,0	4 115	559	27
Glob.	85,9	43 136	7 088	249 → 9

Accord de sens et de montant sur les transactions appariées : ~93 % (deux mesures).

C.5 — Concordance Quiver Sénat, par an (clé normalisée, reconcile).

An	couv. %	sens %	date %	montant %	amend. dédup.	only_quiver
2020	99,4	95,7	99,4	93,2	144	8
2021	98,0	87,0	98,0	86,1	75	8
2022	99,4	91,7	99,4	91,5	86	4
2023	99,7	93,0	99,7	89,5	94	2
2024	99,6	95,6	99,6	92,8	111	2
2025	99,8	96,4	99,8	96,1	20	1
2026	100,0	100,0	100,0	99,3	24	0
Tot.	98–100	~95,7	~99,4	~93,2	554	0 vrai raté

C.6 — Fiabilité des dates OCR (date_confidence, lignes OCR seulement).

An	Chambre (OCR)		Sénat (OCR)	
	plausible %	implausibles	plausible %	implausibles
2020	98,6	120	96,1	9
2021	79,6	1 389	97,5	6
2022	96,4	394	99,5	0
2023	99,2	50	100,0	0
2024	96,5	220	97,6	0
2025	98,1	118	99,0	3
2026	100,0	0	99,7	0
Tot.	95,3	2 291	98,5	18

Le creux 2021 Chambre est entièrement Diana Harshbarger (2 documents manuscrits denses).

C.7 — Clusters OCR & concentration.

Clusters Chambre (census 547 scans)	Concentration des déposants OCR
A — tapé droit : 74 docs	Chambre : Khanna 30 862 (62,9 %), McCaul 10 876 (22,2 %),
B — tapé tourné : 322 docs	Harshbarger 3 514 (7,2 %) → top 3 = 92,4 %, top 10 = 99,0 %
C — manuscrit : 151 docs (exclu)	Sénat : Blumenthal 1 233 (73 %), Boozman 379, Burr 52,
	Feinstein 14, Fetterman 2

C.8 — Statut de validation par déposant (Chambre, crosscheck).

Statut	dépôts	transactions	dont OCR
quiver_validable (Quiver le couvre)	212	78 530	47 004
ocr_unique (source unique : Quiver = 0, OCR $\geq$ 50 %)	13	1 957	1 949
digital (peu / pas d'OCR)	31	1 155	13

Identité : Chambre 256 bioguides / 275 noms (99,99 %), Sénat 64 / 67 (100 %). Golden (zéro écart) : 108 fichiers Chambre, 69 Sénat. **Total : 90 487 transactions**, 2020–2026.

12.5 Annexe E — reproduire / lancer

```
# Installation
python3.12 -m venv .venv && ./venv/bin/pip install -e .
```

```

# Pipeline complet (reseau + API Vision requis) ; --dry-run = voir la sequence
./venv/bin/python -m congress_core.pipeline --years 2020-2026
./venv/bin/python -m congress_core.pipeline --years 2024 --dry-run

# Rapport de qualite (offline, lecture seule des FINAL)
#   -> docs/RAPPORT_QUALITE.md + docs/quality/*.png
./venv/bin/python -m congress_core.quality

# Filet "zero changement" (doit afficher ZERO ECART)
./venv/bin/python tests/regression/check_golden.py           # Chambre, 108
./venv/bin/python tests/regression/senate_check_golden.py    # Senat,      69
./venv/bin/python tests/regression/test_senate_repro.py      # reproductions

```

Document dérivé et vérifié sur le code source réel et les tables FINAL du dépôt. Toute divergence avec une autre documentation doit être tranchée en faveur du code et des données. Quiver = vérification externe, jamais réinjecté.